

LLM-based Compatibility Detection

xxxxxxx[1]

1 Introduction

2 Background

2.1 API Compatibility Detection

The research necessity of API compatibility detection is driven by the combination of practical pain points of developers, technical evolution challenges, industry value demands, and gaps in existing solutions. In open-source communities such as GitHub, a large number of developer complaints indicate that version changes of third-party dependency package APIs often cause project errors—for example, runtime errors resulting from interface parameter adjustments or method deprecation. These issues not only increase debugging costs but also may disrupt project iteration. The high-frequency occurrence of such problems in open-source projects directly reflects the urgent need for efficient detection tools. With the popularization of microservices and multi-cloud architectures, systems have become increasingly dependent on third-party APIs; a single project may involve dozens or even hundreds of API versions. Mixed scenarios involving cross-protocols (e.g., REST, gRPC) and multi-data formats (e.g., JSON, PROTOBUF) further exacerbate the difficulty of version management and compatibility verification. Traditional manual detection or mainstream tools (such as Postman and Swagger) focus on functional testing, making it difficult to cover implicit compatibility issues (e.g., semantic-level differences, residual implicit dependencies) and unable to meet detection needs in complex architectures. From the perspective of industry value, API compatibility has become a core element in ensuring system stability—in critical fields such as finance and healthcare, compatibility issues may lead to severe consequences such as transaction interruptions and data errors, resulting in direct economic losses. In cross-enterprise collaboration, version incompatibility prolongs the system integration cycle and reduces the efficiency of ecological collaboration. However, existing technologies lack sufficient capability to analyze API semantic differences and struggle to address detection needs in scenarios involving dynamically generated APIs and multi-role usage. This highlights the research value of detection solutions based on LLMs (Large Language Models) combined with COT (Chain of Thought) and MultiAgent technologies: through logical reasoning to analyze the semantic impact of changes and multi-perspective simulation for collaborative verification, these solutions are expected to break through the limitations of traditional schemes, solve the practical pain points of developers and technical bottlenecks in complex architectures, and hold significant theoretical and practical significance for ensuring system reliability and improving industry collaboration efficiency.

2.2 API Compatibility Type

2.3 Motivating Example

3 Approach

This section details our methodology for detecting Python API incompatibilities using Large Language Models (LLMs) with structured Chain-of-Thought (COT) prompting. We first outline the overall workflow and core challenges, then elaborate on the design principles compared to existing methods, and technical implementation

3.1 CoT

3.2 Overview and Challenges

Our approach targets code-level changed API pairs (excluding module-level, class-level, and API addition/deletion changes) between two versions of a Python project. Figure x illustrates the end-to-end workflow. First, we extract all APIs from two versions of the project and filter API pairs to retain only those with code-level modifications. Then we instruct LLM with COT guidance to performs line-by-line code analysis. Finally, The LLM generates a structured JSON output, including detected incompatibility types, detailed reasons for each type, and auxiliary information.

figure x

During this three-step process, we need to address the following three challenges:

- **High-Level Description Misalignment (C1):** Existing LLM-based methods[x] often rely on high-level incompatibility descriptions rather than code-level semantics. This disconnect causes LLMs to struggle with mapping abstract descriptions to concrete code changes.
- **Scalability with Incompatibility Categories (C2):** As the number of incompatibility categories increases, naive prompting leads to exponential growth in prompt token size and LLM runtime. For N categories, this approach requires N independent LLM calls per API pair, making it infeasible for large-scale analysis.
- **Unreliable and Inconsistent Outputs (C3):** LLMs exhibit inherent output randomness—they may produce contradictory judgments for the same prompt (e.g., labeling a parameter rename as “compatible” in one run and “incompatible” in another) or include internally inconsistent reasoning (e.g., citing non-existent code lines to support a claim).
- **Unstructured COT Reasoning (C4):** Without guided thinking frameworks, LLMs’ intermediate reasoning (e.g., how they identify a parameter default value change) remains unstructured. This makes it impossible to verify the validity of the LLM’s conclusions.

3.3 Methodology Design

To address the above challenges, we design a one-pass COT analysis framework with structured prompting, built on insights from prior empirical studies and innovations in LLM guidance.

We first establish a comprehensive taxonomy of Python API incompatibilities, extending the 7-category framework proposed by Shen et al. [x] (a foundational empirical study on Python API evolution). Shen et al. identified incompatibilities rooted in parameters (CI1-CI5), exceptions (CI6), and return values (CI7). Through our own data collection, we validated these 7 categories and discovered two new, previously unreported types:

CI8: global variable change **CI9:** reference variable change

Table x

3.3.1 Limitation of Current Methods. Traditional static analysis methods (e.g., Shen et al. [2024]) rely on handcrafted rules. However, Rule design requires expert knowledge making it hard to generalize to new libraries or unknown incompatibilities. For example, shen define data structure P to record parameter information for the detection of CI1-CI5. However, without detailed study into incompatible examples. it is hard to design rules for detections. Besides, These methods fail to interpret semantic context while many crucial infomation should be analyzed through variable names, comments, usage in function bodies, etc. For example, Shen et al.’s method cannot recognize that renaming a parameter

from arguments to args is a compatibility issue—since the AST pattern for parameter names differs, but the static rule does not link the semantic similarity between arguments and args.

In our method, by leveraging LLMs’ semantic understanding capabilities, our approach directly addresses this limitation. The LLM integrates contextual cues to identify semantic changes, even when syntactic patterns are non-trivial.

3.3.2 One-Pass COT Analysis Framework. To tackle scalability and output unreliability, we design a one-pass COT framework guided by two core principles: two-stage analysis and unified scanning. First, the LLMs is instructed to process the parameter list, which is syntactically structured, then analyze the function body, which is semantically rich—aligning with how developers naturally review code changes. We embed rules for all 9 incompatibility categories into a single prompt to enable the LLM to detect multiple incompatibilities in one analysis pass, avoiding repeated LLM calls. The framework is divided into four sequential phases, each with predefined data structures and rules to guide the LLM’s reasoning. This design offers three key advantages: efficiency, as runtime is independent of the number of incompatibility categories (all rules are processed in one pass); reduced misjudgment, as it enforces dependencies between interdependent categories (e.g., analyzing CI1 (parameter addition) before CI4 (default value change) to avoid misclassifying added parameters as default value changes); and structured reasoning, as each phase outputs intermediate data structures like parameter mappings and exception type lists, which make the LLM’s logic traceable and verifiable.

3.3.3 Structured Prompt Design. The prompt is the backbone of our framework, structured to enforce consistency and guide the LLM’s COT reasoning. It includes four components: Instructions, Analysis Phases, Source Code, and Examples. A simplified template is shown below (full rules are provided in Appendix A):

Analyze two versions of a Python API function (`code\_v1` and `code\_v2`) to detect incompatibilities. The output must be in JSON format.

```
### Instructions
Please strictly follow the `Analysis Phases` to identify all
incompatibility types (CI1 - CI9) and provide concise reasons. The
final output must include:
- `additional_info`: Key variables/structures used in analysis (e.g.,
  parameter mappings).
- `reasons`: One-sentence explanations linking code changes to
  incompatibility types.
- `incompatibility`: List of detected types

### Analysis Phases
#### Phase 0: Initialization
Initialize the result dictionary:
{
  "incompatibility": [], "reasons": [], "additional_info": {"param_mapping":
    {}}, "exception_list_v1": [], ...}
```

```

}
#### Phase 1: Parameter List Processing
Extract parameter lists from `code_v1` and `code_v2`, then build:
#### Phase 2: Initial Incompatibility Analysis
Use `P1`/`P2` to update results:
#### Phase 3: Function Body Processing (Line-by-Line)
Initialize data structures for `code_v1`/`code_v2`:
#### Phase 4: Comprehensive Incompatibility Analysis
Use Phase 1-3 data structures to detect remaining types:

### Source Code
"code_v1": ...
"code_v2": ...

### Examples
}

```

3.3.4 Data Structure and Detection Design. The analysis leverages several core data structures to capture critical information from function signatures and bodies, facilitating subsequent incompatibility checks. Based on the detecting process for different incompatibility types, we design following data structure which is initialized at beginning and update throughout the scanning of the source code.

Parameter Lists (P1, P2): These lists encode the parameters of source code. Each parameter is represented as a tuple (name, position index, type, default value), where type includes positional, optional (with a default value), or kwargs, enabling direct comparison of parameter presence, order, and defaults.

KWARGS: A string storing the name of the kwargs parameter if present in both code v1 and code v2; empty otherwise. This structure anchors the analysis of keyword argument keys accessed within the function body.

ReName: A list of tuples (name1, name2): name1 (from code v1) and name2 (from code v2) are semantically similar or share the same position index, indicating potential parameter renaming.

Keyword Argument Keys (K1, K2): Lists of keys explicitly accessed via kwargs (e.g., kwargs.get('key') or kwargs['key']) in code v1 and code v2, respectively. These capture dependencies on dynamic keyword arguments.

Renamed Parameter Usage (ReNUsage1, ReNUsage2): Dictionaries mapping parameters in ReName pairs to lists of code lines where they are used. This tracks contextual usage to validate renaming.

Condition Statements (C1, C2): Lists of condition strings (e.g., if x > 0) extracted from code v1 and code v2, indexed sequentially to link with dependent logic (e.g., assignments, exceptions).

Data Flow (D1, D2): Lists of tuples (variable name, variable type, dependent variables, condition indices) capturing variable assignments. variable type distinguishes local variable, global variable, and reference variable (e.g., self), while dependent variables and condition indices track dependencies and associated conditions.

Exceptions (E1, E2): Lists of tuples pairing explicit exception types (e.g., ValueError) with indices of conditions under which they are raised (from C1/C2), excluding implicit behaviors or unthrown exceptions.

Return Values (R1, R2): Lists of tuples pairing return expressions with condition indices, capturing how return values vary under different conditions.

The 9 types of incompatibility can be detected based on the data structure above. The detection rules are showed in Table x.

Table x

3.4 implementation

We implement the approach using Python, leveraging open-source tools and APIs to balance reproducibility and efficiency. For API extraction, we combine the GitHub API and Python’s standard ast library: the GitHub API fetches source code for specific project versions to enable automated data collection, while the ast library builds ASTs for function-level slicing—extracting only the function body (excluding imports or global statements outside the API). For LLM interaction, we use DeepSeek-chat API with temperature 0 to minimize randomness and max-tokens 2048 to accommodate structured outputs.

3.5 Multi-Agent

4 Evaluation

4.1 Dataset Construction

Dataset A.

Dataset B.

4.2 Experiment Setup

- **Accuracy Evaluation (RQ1):** How is the overall accuracy of TOOL compared with the state-of-the-arts?
- **Effectiveness Evaluation (RQ2):** How is the effectiveness of TOOL in different API compatibility types?
- **LLM Generalizability (RQ3):** How is the accuracy of TOOL using different LLMs?
- **Ablation Study (RQ4):**

Baselines. We selected XXX [2], XXX [2], and XXX [2] as the comparison.

Metrics. We use precision and recall as metrics, which are defined as follows:

4.3 Results

4.3.1 Accuracy Evaluation (RQ1).

4.3.2 RQ2.

4.3.3 RQ3.

4.3.4 RQ4.

5 Related Work

5.1 Traditional Compatibility Detection Methods

API compatibility of Python third-party libraries is a critical concern given their widespread use in diverse domains, and relevant detection methods have been a focus of research.

Early work in this space focused on static analysis to model API lifecycles, but with limited granularity. Quan et al. (2022) pioneered this direction for Python with PyMevol, a static tool that constructs a history-sensitive, alias-aware API explorer tree. PyMevol effectively tracked API introductions, modifications, and removals, and addressed Python’s unique transitive import-induced alias problem—an issue overlooked by earlier static tools for statically typed languages. However, PyMevol’s focus on high-level API lifecycle tracking left a gap in identifying fine-grained causes of incompatibilities (e.g., parameter range changes, exception type shifts). This limitation was addressed by Shen et al., who proposed a more precise static analysis method: by leveraging statement-level dataflow and control-flow slicing, their work extracted detailed incompatibility causes (e.g., parameter additions/deletions, default value modifications, return type changes) that PyMevol could not capture, thereby enhancing the actionable insights for developers.

While static analysis methods (Quan et al., Shen et al.) excelled at single-project syntactic detection, they struggled to uncover semantic-level incompatibilities across multiple client projects—a critical scenario for real-world library usage. Chen et al. (2020) addressed this gap with DeBBI, which transformed behavioral backward incompatibility (BBI) detection into an information retrieval problem. Unlike PyMevol and Shen et al.’s single-project focus, DeBBI prioritized client projects with high relevance to API changes (via TF-IDF and LDA models) and optimized test execution via regression test selection. This approach not only resolved the inefficiency of large-scale cross-project testing but also detected semantic BBIs (e.g., undocumented behavioral shifts) that static syntactic analysis missed.

There are also some studies focusing on conflicts between repositories (e.g., Python’s PyPI and Ubuntu’s apt). Xie et al. tackled this with Hera, a tool that built an offline cross-repository compatibility database. Hera addressed the cross-repository gap by modeling dependencies across PyPI and apt, and could predict, detect, and fix conflicts arising from mixed package managers—an issue that single-repository methods were unable to handle.

In domain-specific contexts, Zhao et al. extended compatibility detection to deep learning (DL) projects, a space where general-purpose methods (e.g., DeBBI, Hera) struggled due to DL’s complex stack (libraries, runtime, drivers, OS, hardware). Zhao et al. constructed a weighted knowledge graph from Stack Overflow discussions to capture DL-specific incompatibilities, but this method relied on pre-extracted, manually validated knowledge, lacking dynamic semantic reasoning. Similarly, Liu et al. conducted a comparative study of nine Android API compatibility tools, revealing fragmentation in tool capabilities across device-specific and evolution-induced issues, but like Zhao et al.’s work, it did not leverage advanced semantic models to enhance detection. Even Zhang et al.’s targeted work on Python variadic parameter pitfalls—which supplemented general static analysis with language-specific insights—remained confined to syntactic pattern matching.

Across all these traditional methods, a unifying limitation persisted: none leveraged Large Language Models (LLMs) for advanced semantic analysis. These approaches failed to harness LLMs’ strengths in contextual reasoning and nuanced semantic understanding. In contrast, our work addresses this critical gap by integrating LLMs, enhanced via Chain-of-Thought (CoT) reasoning and multi-agent collaboration. This approach capitalizes on LLMs’ ability to interpret implicit API behaviors, infer context-dependent compatibility rules, and adapt to diverse usage scenarios—capabilities that resolve the semantic detection limitations of all preceding traditional methods.

5.2 LLMs in the Context of API Evolution

Research on LLM-driven reasoning enhancement and practical task adaptation has yielded insights directly relevant to API evolution scenarios, with a clear trajectory of addressing prior limitations to improve adaptability and precision.

Building on the foundational work of Wei et al. (2022), who pioneered Chain-of-Thought (CoT) prompting, LLMs gained the ability to perform multi-step reasoning—an essential capability for untangling the complex logic of API

evolution (e.g., tracking parameter changes, dependency shifts). However, CoT’s unstructured reasoning steps often led to ambiguity in code-related tasks, a gap addressed by Li et al. (2025) with their Structured CoT (SCoT) prompting. By integrating programming’s core structures (sequential, branch, loop) into reasoning steps, SCoT significantly boosted code generation accuracy, directly supporting code adaptation in dynamically evolving API environments where structural consistency is critical. Complementing SCoT’s structural optimization, Zhang et al. (2025) identified another limitation of CoT: its reliance on semantically similar demonstrations, which hinders robustness across diverse reasoning scenarios. To resolve this, they proposed Pattern-CoT, which extracts task-specific reasoning patterns (e.g., arithmetic operations, logical judgments) to select diverse demonstrations. This approach enhanced LLMs’ ability to generalize across varied API evolution contexts, such as handling both parameter adjustments and return type modifications.

While structural and demonstration-related optimizations advanced CoT, Liu et al. (2024) noted a lack of focus on relational understanding—key for tracking associated modifications in API evolution (e.g., how a change in one API affects dependent methods). Their ERA-CoT framework leverages entity relationship analysis to strengthen LLMs’ grasp of relational changes, filling this gap and improving the detection of cascading impacts in evolving API ecosystems.

In practical scenarios tied to API evolution, such as vulnerability detection induced by API changes, Ma et al. (2024) developed iAudit, a framework combining LoRA fine-tuning and LLM-based agents (Ranker/Critic) for smart contract auditing. Similarly, Sun et al. (2024) proposed GPTScan, which fuses GPT’s code understanding with static analysis to detect logic vulnerabilities in smart contracts—both works demonstrate the value of LLM integration with specialized analysis for evolution-related risk detection. However, neither targets API compatibility directly, focusing instead on general vulnerability auditing.

A critical limitation across these LLM-based approaches was highlighted by Stechly et al. (2024): CoT and its variants rely heavily on task-specific prompts, lacking adaptability in dynamic API evolution scenarios where requirements (e.g., Python’s variadic parameter changes, version-specific trait shifts) are constantly evolving. More notably, none of these studies design prompts tailored to API compatibility analysis, nor do they prioritize the unique characteristics of Python APIs (e.g., dynamic imports, alias handling, variadic parameters)—a key gap given Python’s dominance in third-party library ecosystems. To address these shortcomings, our work crafts API compatibility-focused prompts that integrate Python-specific API traits (e.g., version-dependent parameter constraints, variadic parameter behaviors) and optimizes LLM reasoning for Python’s dynamic features. By combining CoT’s multi-step reasoning with Python API evolution insights and multi-agent collaboration, we enable precise semantic analysis of compatibility issues—filling the void of LLM-based solutions tailored to Python API compatibility detection.

References

- [1] Vahid Garousi and Mika V Mäntylä. 2016. A systematic literature review of literature reviews in software testing. *Information and Software Technology* 80 (2016), 195–216.
- [2] Zaixi Zhang, Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. 2022. Fldetector: Defending federated learning against model poisoning attacks via detecting malicious clients. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2545–2555.